
# Software Requirements Document

## Songify

**Software Requirements Specification**
for
**SONGIFY**
Version 1.0 Approved
Prepared by: **Tahawar, Faizan, Ahmad**
Organization: **University Project**
Date: **March 2025**

---

# Table of Contents

6. Other Requirements
   Appendix A: Glossary
   Appendix B: Analysis Models
   Appendix C: To Be Determined List

---

# 1. Introduction

## 1.1 Purpose

The purpose of this document is to define the software requirements for **SONGIFY**. The system will provide essential music playback features along with additional functionalities such as library management, offline playback, and optional AI-based recommendations.

## 1.2 Document Conventions

- **Bold text** represents section headings.
- `Code formatting` is used for technical commands or references.
- Requirements will be uniquely numbered (e.g., REQ-1, REQ-2).

## 1.3 Intended Audience and Reading Suggestions

This document is intended for software developers, project managers, UI/UX designers, and testers involved in the development of the application.

## 1.4 Product Scope

SONGIFY will enable users to play, manage, and organize music efficiently. It includes support for offline playback, customizable UI, and optional AI integration for personalized recommendations.

## 1.5 References

- Android Developer Documentation
- IEEE SRS Standard

# 2. Overall Description

## 2.1 Product Perspective

The application will be a standalone **Android mobile app**, leveraging Android's **MediaPlayer** or **ExoPlayer** for audio playback.

### 2.2 Product Functions

- **Basic Music Playback** (play, pause, seek, shuffle, repeat)
- **Library Management** (browse music by albums, artists, genres)
- **Offline Playback** (play downloaded music)
- **User Interface** (intuitive design, dark/light themes)
- **Notifications & Widgets** (control music from notifications)
- **Optional AI-based recommendations**

### 2.3 User Classes and Characteristics

- **General Music Listeners**: Users looking for a simple, efficient music player.
- **Power Users**: Users who organize music by metadata and require advanced playlist features.

### 2.4 Operating Environment

- OS: Android (API level 21+)
- Programming Language: Kotlin
- Backend (for AI Features): TensorFlow Lite (optional)

### 2.5 Design and Implementation Constraints

- Must follow Android UI/UX guidelines.
- Performance-optimized for low-end devices.

### 2.6 User Documentation

- In-app tutorial.
- Online help section.

### 2.7 Assumptions and Dependencies

- Users will provide their own music files.
- AI-based features require internet connectivity.

## 3. External Interface Requirements

### 3.1 User Interfaces

- Material Design UI with dark and light themes.

### 3.2 Hardware Interfaces

- Supports headphone controls.

### 3.3 Software Interfaces

- Integrates with Android MediaPlayer or ExoPlayer.

### 3.4 Communications Interfaces

- AI-based recommendations require API access (if implemented).

## 4. System Features

### 4.1 Basic Music Player Features

**Description:** Play, pause, forward, and backward controls with volume and seek functionality.

**Functional Requirements:**

- REQ-1: System must allow users to play and pause audio.
- REQ-2: System must support shuffle and repeat modes.

### 4.2 Music Library Management

**Description:** Organizing music by albums, artists, and genres.

**Functional Requirements:**

- REQ-3: Users must be able to browse and search their music library.

### 4.3 User-Friendly Interface

**Description:** Clean and customizable UI/UX design.

**Functional Requirements:**

- REQ-4: System must provide both light and dark themes.

### 4.4 Offline Playback

**Description:** Users can play downloaded music without an internet connection.

**Functional Requirements:**

- REQ-5: System must allow playback of locally stored files.

### 4.5 Notifications and Widgets

**Description:** Display currently playing song in the notification bar and provide quick access controls.

**Functional Requirements:**

- REQ-6: System must display media controls in notifications.

### 4.6 Optional AI Integration (Stretch Goal)

**Description:** Suggest songs based on user listening history and preferences.

**Functional Requirements:**

- REQ-7: If implemented, the AI must analyze user preferences for recommendations.

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

- The system should handle large music libraries efficiently.

### 5.2 Safety Requirements

- App must not interfere with other system audio functions.

### 5.3 Security Requirements

- User data must not be shared with third parties.

### 5.4 Software Quality Attributes

- **Usability:** Intuitive and responsive design.
- **Maintainability:** Modular architecture for future updates.

### 5.5 Business Rules

- No DRM-restricted content should be playable.

## 6. Other Requirements

- AI recommendation system is optional.

## Appendices

## Appendix A: Glossary

- **ExoPlayer:** An Android media player library
- **TensorFlow Lite:** A machine learning framework for mobile AI

## Appendix B: Analysis Models

- UI wireframes (to be included later)

## Appendix C: To Be Determined List

- AI model implementation feasibility.
- Final design for music browsing UI.

---

This document serves as the foundation for system development. Feedback and revisions will be incorporated as necessary.